

Rapport TD n°1 : Programmation impérative

Derrez Lorenzo

September 11, 2026

Exercice 0

Listing 1: Print Hello World

```
1  #include <stdio.h>
2  int main(int argc, char* argv[]){
3      printf("Hello_world\n");
4      return 0;
5  }
6  // Output : Hello world
```

Exercice 1

Listing 2: PGCD

```
1  // Q1 :
2  int pgcd_rec(int a, int b){
3      if (b == 0){
4          return a;
5      }
6      else{
7          int r = a%b;
8          return pgcd_rec(b, r);
9      }
10 }
11
12 //Q3 :
13 int pgcd_iter(int a, int b){
14     ///
15 }
16
17 // Test :
18 int main(int argc, char* argv[]){
19     int x1 = 15;
20     int x2 = 3;
21     int t1 = pgcd_rec(x1, x2);
22     printf("Test_1_result_:_%d\n", t1);
23
24     int y1 = 4;
25     int y2 = 8;
26     int t2 = pgcd_rec(y1, y2);
27     printf("Test_2_result_:_%d\n", t2);
28
29     int z1 = 10;
```

```

31     int z2 = 0;
32     int t3 = pgcd_rec(z1, z2);
33     printf("Test_3_result_:_%d\n", t3);
34     return 0;
35 }

```

Exercise 2

Listing 3: Array Exercise

```

1
2 void affiche_tab(int* tab, int n){
3     int i = 0;
4     while(i<n){
5         printf("%d_", tab[i]);
6         i++;
7     }
8     printf("\n");
9 }
10
11 /// Test
12
13 int main(int argc, char* argv[]){
14     int n = 5;
15     int* tab1 = malloc(n*sizeof(int));
16
17     int i = 0;
18     while(i<n){
19         tab1[i] = i;
20         i++;
21     }
22
23     affiche_tab(tab1, n);
24     return 0;
25 }
26
27 // Q3 :
28
29 int* n_premier_pairs(int n){
30     int* tab = malloc(n*sizeof(int));
31     int cpt = 0;
32
33     int nbr = 0;
34
35     while(cpt!=n){
36         if(nbr%2==0){
37             tab[cpt] = nbr;
38             nbr++;
39             cpt++;
40         }
41         else{
42             nbr++;
43         }
44     }
45     return tab;
46 }
47

```

```

48 // Modification of the main function
49
50 int main(int argc, char* argv[]){
51     int n = 5;
52     int* tab1 = n_premier_pairs(n);
53
54     affiche_tab(tab1,n);
55
56     return 0;
57 }

```

Exercice 3

Listing 4: Exercices Array

```

1
2 // Q1 :
3
4 int sum_elts_array(int* tab, int n){
5     int s = 0;
6     int i = 0;
7     while(i<n){
8         s += tab[i];
9         i++;
10    }
11    return s;
12 }
13
14 // Q2
15
16 int sqr(int x){
17     int i = 0;
18     while(i*i<x){
19         i++;
20     }
21     return i;
22 }
23
24 int produit_scalaire(int* tab1, int* tab2, int n){
25     int i = 0;
26     int res = 0;
27
28     while (i<n){
29         res += tab1[i]*tab2[i];
30         i++;
31     }
32     return sqr(res);
33 }
34
35 // Q3 :
36
37 int somme_memb_memb_arr(int n, int* tab1, int* tab2){
38     int i = 0;
39     int res = 0;
40     while(i<n){
41         res += tab1[i]+tab2[i];
42         i++;

```

```

43     }
44     return res;
45 }
46
47 // Test
48
49 int main(int argc, char* argv[]){
50     int n = 5;
51
52     int tab1[5] = {0, 1, 2, 3, 4};
53     int tab2[5] = {5, 4, 3, 2, 1};
54
55     int t1 = sum_elts_array(tab1, n);
56     printf("Test first function for tab1: %d\n", t1);
57
58     int t2 = produit_scalaire(tab1, tab2, n);
59     printf("Test second function: %d\n", t2);
60
61     int t3 = somme_memb_memb_arr(n, tab1, tab2);
62     printf("Test third function: %d\n", t3);
63
64     return 0;
65 }

```

Exercice 4

Listing 5: Dichotomie

```

1 // Q1 :
2
3 int est_trie(int* tab, int n){
4     int i = 1;
5     int anc_val = tab[0];
6
7     while(i < n && anc_val < tab[i]){
8         anc_val = tab[i];
9         i++;
10    }
11    if (n == i){
12        return 1;
13    }
14    else{
15        return 0;
16    }
17 }
18
19
20 // Q2 :
21
22 int is_elt(int* tab, int n, int x){
23     int i = 0;
24     while(i < n){
25         if (x == tab[i]){
26             return 1;
27         }
28         i++;
29     }

```

```

30     return 0;
31 }
32
33 // Q3
34
35 int is_elem_dich(int* tab, int n, int x){
36     int debut = 0;
37     int fin = n-1;
38
39     while(debut<fin){
40         int mid = (debut+fin)/2;
41         if(tab[mid]==x) return 1;
42         if(x < tab[mid]){
43             fin = mid-1;
44         }
45         else{
46             debut = mid+1;
47         }
48     }
49     return 0;
50 }
51
52 // Q4 :
53 int rech_dich_rec(int* tab, int n, int deb, int fin, int x){
54
55     if (deb >= fin) return 0;
56
57     int mid = (deb+fin)/2;
58     if(tab[mid]==x){return 1;}
59     if(tab[mid]>x){
60         fin = mid-1;
61         return rech_dich_rec(tab, n, deb, fin, x);
62     }
63     else{
64         deb = mid+1;
65         return rech_dich_rec(tab, n, deb, fin, x);
66     }
67 }
68
69 //Test
70
71 int main(int argc, char* argv[]){
72     int n = 5;
73     int tab1[5] = {0, 1, 2, 3, 4};
74     int t1 = est_trie(tab1, n);
75     printf("Test_Q1: %d\n", t1);
76
77     int x1 = 3;
78     int t2 = is_elt(tab1, n, x1);
79     printf("Test_Q2: %d\n", t2);
80
81     int x2 = 8;
82     int t3 = is_elem_dich(tab1, n, x2);
83     printf("Test_Q3: %d\n", t3);
84
85     int x3 = 7;
86     int t4 = rech_dich_rec(tab1, n, 0, 4, x3);
87     printf("Test_Q4: %d\n", t4);
88 }

```

Exercice 5

Listing 6: Binomial coefficients

```
1 // Q1
2
3 void pascal_ligne(int n, int* valeurs){
4     int k = 1;
5
6     printf("%d_", 1);
7
8     while (k<n-1){
9         int elt = valeurs[k-1] + valeurs[k];
10        printf(" %d_", elt);
11        k++;
12    }
13    printf("1_\n");
14 }
15
16
17 // Q2
18
19 void affiche_pascal(int n, int* valeurs, int i){
20     if(i<n){
21         int ligne_i[i+1];
22         int k = 1;
23         ligne_i[0] = 1;
24         ligne_i[i] = 1;
25         printf("1_");
26         while(k<i){
27             int elt = valeurs[k-1]+valeurs[k];
28             ligne_i[k] = elt;
29             printf("%d_", ligne_i[k]);
30             k++;
31         }
32         printf("1_\n");
33         i+=1;
34         affiche_pascal(n, ligne_i, i);
35     }
36     printf("\n");
37 }
38
39 int main(int argc, char* argv[]){
40     //Q3
41     int n;
42
43     printf("Combien de lignes voulez-vous? ");
44     scanf("%d", &n);
45
46     int valeurs[2] = {1,1};
47     pascal_ligne(n, valeurs);
48
49     int i = 1;
50     int val1[1] = {1};
51     printf("1_\n");
52     affiche_pascal(n, val1, i);
53
54     return 0;
55 }
```

Exercice 6

Listing 7: Fibonacci

```
1 // Q1
2 int fibo_rec(int x){
3     if(x<1)return 0;
4     if(x==1)return 1;
5     if(x>1){
6         return fibo_rec(x-1)+fibo_rec(x-2);
7     }
8     // Return -1 pour respecter la signature de la fonction fibo_rec
9     return -1;
10 }
11
12 // Q2 Main after Q3
13
14 // Q3
15
16 int fibo_iter(int x, int* memo){
17     memo[0] = 0;
18     memo[1] = 1;
19
20     int i = 2;
21     while(i<=x){
22         memo[i] = memo[i-1]+memo[i-2];
23         i+=1;
24     }
25     return memo[x];
26 }
27
28 // Q4
29 // We have to add date_time to determinate during the execution thanks to main
    function which method is the fastest
30 #include <time.h>
31
32 int main(int argc, char* argv[]){
33
34     int x;
35     int resultat_rec;
36     int resultat_iter;
37
38     clock_t debut;
39     clock_t fin;
40
41     double temps_rec;
42     double temps_iter;
43
44     int memo[46];
45
46
47     // Test avec x = 10
48     x = 10;
49
50     debut = clock();
51     resultat_rec = fibo_rec(x);
52     fin = clock();
53
54     temps_rec = (double)(fin - debut) / CLOCKS_PER_SEC;
```

```

55
56 debut = clock();
57 resultat_iter = fibo_iter(x, memo);
58 fin = clock();
59
60 temps_iter = (double)(fin - debut) / CLOCKS_PER_SEC;
61
62 printf("x=%d\n", x);
63 printf("fibo_rec: %d\n", resultat_rec);
64 printf("temps: %f secondes\n", temps_rec);
65 printf("fibo_iter: %d\n", resultat_iter);
66 printf("temps: %f secondes\n\n", temps_iter);
67
68
69 // Test avec x = 20
70 x = 20;
71
72 debut = clock();
73 resultat_rec = fibo_rec(x);
74 fin = clock();
75
76 temps_rec = (double)(fin - debut) / CLOCKS_PER_SEC;
77
78 debut = clock();
79 resultat_iter = fibo_iter(x, memo);
80 fin = clock();
81
82 temps_iter = (double)(fin - debut) / CLOCKS_PER_SEC;
83
84 printf("x=%d\n", x);
85 printf("fibo_rec: %d\n", resultat_rec);
86 printf("temps: %f secondes\n", temps_rec);
87 printf("fibo_iter: %d\n", resultat_iter);
88 printf("temps: %f secondes\n\n", temps_iter);
89
90
91 // Test avec x = 30
92 x = 30;
93
94 debut = clock();
95 resultat_rec = fibo_rec(x);
96 fin = clock();
97
98 temps_rec = (double)(fin - debut) / CLOCKS_PER_SEC;
99
100 debut = clock();
101 resultat_iter = fibo_iter(x, memo);
102 fin = clock();
103
104 temps_iter = (double)(fin - debut) / CLOCKS_PER_SEC;
105
106 printf("x=%d\n", x);
107 printf("fibo_rec: %d\n", resultat_rec);
108 printf("temps: %f secondes\n", temps_rec);
109 printf("fibo_iter: %d\n", resultat_iter);
110 printf("temps: %f secondes\n\n", temps_iter);
111
112
113 // Test avec x = 40

```

```

114     x = 40;
115
116     debut = clock();
117     resultat_rec = fibo_rec(x);
118     fin = clock();
119
120     temps_rec = (double)(fin - debut) / CLOCKS_PER_SEC;
121
122     debut = clock();
123     resultat_iter = fibo_iter(x, memo);
124     fin = clock();
125
126     temps_iter = (double)(fin - debut) / CLOCKS_PER_SEC;
127
128     printf("x=%d\n", x);
129     printf("fibo_rec: %d\n", resultat_rec);
130     printf("temps: %f secondes\n", temps_rec);
131     printf("fibo_iter: %d\n", resultat_iter);
132     printf("temps: %f secondes\n\n", temps_iter);
133
134
135     return 0;
136 }
137
138 //x = 10
139 //fibo_rec : 55
140 // temps : 0.000002 secondes
141 // fibo_iter : 55
142 // temps : 0.000001 secondes
143
144 // x = 20
145 // fibo_rec : 6765
146 // temps : 0.000061 secondes
147 // fibo_iter : 6765
148 // temps : 0.000001 secondes
149
150 // x = 30
151 // fibo_rec : 832040
152 // temps : 0.008850 secondes
153 // fibo_iter : 832040
154 // temps : 0.000001 secondes
155
156 // x = 40
157 // fibo_rec : 102334155
158 // temps : 1.051600 secondes
159 //fibo_iter : 102334155
160 //temps : 0.000001 secondes

```

Exercice 7 : Bonus

Listing 8: Exo nombres de comparaisons tests

```

1     //// Exercice 7 : Bonus ////
2
3 // Q1 : sieve of Eratosthenes
4
5 // Conjecture : tabool has a length of n+1. With that information, the value is the
   index

```

```

6
7 int sqr(int x){
8     int i = 0;
9     while(i*i<x){
10         i++;
11     }
12     return i;
13 }
14
15 void eratosthene(int* tabool, int n){
16     int pp_entier = 2;
17     int pg_entier = sqr(n);
18
19     while(pp_entier <= pg_entier){
20         int cpt = pp_entier+1;
21         while(cpt<=n){
22             if(cpt%pp_entier == 0){
23                 tabool[cpt] = 0;
24             }
25             cpt +=1;
26         }
27         pp_entier+=1;
28     }
29 }
30
31 // Test Q1
32
33 // Aux. function
34 void affiche_datab(int* tab, int n){
35     int indice=0;
36     while(indice<=n){
37         printf("%d_", tab[indice]);
38         indice +=1;
39     }
40     printf("\n");
41 }
42
43
44 // int main(int argc, char* argv[]){
45 //     int n = 30;
46
47 //     int* tabool = malloc((n+1)*sizeof(int));
48 //     tabool[0] = 0;
49 //     tabool[1] = 0;
50 //     int i = 2;
51 //     while(i<=n){
52 //         tabool[i] = 1;
53 //         i+=1;
54 //     }
55
56 //     eratosthene(tabool, n);
57 //     affiche_datab(tabool, n);
58
59 //     return 0;
60 // }
61
62 // Q2
63
64 int eratosthene_2_sans_comp(int* tabool, int n){

```

```

65     int pp_entier = 2;
66     int pg_entier = sqr(n);
67
68     // Q2
69     int nb_acc = 0;
70
71     while(pp_entier <= pg_entier){
72         int cpt = pp_entier+1;
73         while(cpt<=n){
74             if(cpt%pp_entier == 0 ){
75                 taboo[cpt] = 0;
76                 nb_acc+=1;
77             }
78
79             cpt +=1;
80         }
81         pp_entier+=1;
82     }
83
84     return nb_acc;
85 }
86
87 int eratosthene_2_avec_comp(int* taboo, int n){
88     int pp_entier = 2;
89     int pg_entier = sqr(n);
90
91     // Q2
92     int nb_acc = 0;
93
94     while(pp_entier <= pg_entier){
95         int cpt = pp_entier+1;
96         while(cpt<=n){
97             if(taboo[cpt]==1 && cpt%pp_entier == 0 ){
98                 taboo[cpt] = 0;
99                 nb_acc+=1;
100             }
101             nb_acc+=1;
102             cpt +=1;
103         }
104         pp_entier+=1;
105     }
106
107     return nb_acc;
108 }
109
110 // Test Q2
111
112 #include <time.h>
113
114 int main(int argc, char* argv[]){
115     int n = 30000;
116
117     int* taboo = malloc((n+1)*sizeof(int));
118     taboo[0] = 0;
119     taboo[1] = 0;
120     int i = 2;
121     while(i<=n){
122         taboo[i] = 1;
123         i+=1;

```

```

124 }
125
126 clock_t debut;
127 clock_t fin;
128
129 double temps_wit_comp;
130 double temps_avec_comp;
131
132 debut = clock();
133 int t1 = eratosthene_2_sans_comp(tabool, n);
134 fin = clock();
135
136 temps_wit_comp = (double)(fin - debut) / CLOCKS_PER_SEC;
137
138 debut = clock();
139 int t2 = eratosthene_2_avec_comp(tabool, n);
140 fin = clock();
141
142 temps_avec_comp = (double)(fin-debut)/CLOCKS_PER_SEC;
143
144 printf("Nombre d'accès sans comparaisons : %d\n", t1);
145 printf("Nombre d'accès avec comparaisons : %d\n", t2);
146
147 printf("Temps sans comparaisons : %f\n", temps_wit_comp);
148 printf("Temps avec comparaisons : %f\n", temps_avec_comp);
149
150
151 return 0;
152 }
153
154 // Q3
155 int eratosthene_3_sans_comp(int* tabool, int n){
156     int nb_op = 0;
157
158     int pp_entier = 2;
159     int pg_entier = sqr(n);
160
161     nb_op +=2;
162
163     while(pp_entier <= pg_entier){
164         int cpt = pp_entier+1;
165         while(cpt<=n){
166             if(cpt%pp_entier ==0 ){
167                 tabool[cpt] = 0;
168                 nb_op+=1;
169             }
170             cpt +=1;
171             nb_op+=4;
172         }
173         pp_entier+=1;
174         nb_op+=3;
175     }
176
177     return nb_op;
178 }
179
180 int eratosthene_3_avec_comp(int* tabool, int n){
181     int nb_op = 0;
182

```

```

183     int pp_entier = 2;
184     int pg_entier = sqr(n);
185
186     nb_op +=2;
187
188     while(pp_entier <= pg_entier){
189         int cpt = pp_entier+1;
190         while(cpt<=n){
191             if(tabool[cpt]==1 && cpt%pp_entier == 0 ){
192                 tabool[cpt] = 0;
193                 nb_op+=1;
194             }
195             cpt +=1;
196             nb_op+=5;
197         }
198         pp_entier+=1;
199         nb_op +=3;
200     }
201
202     return nb_op;
203 }
204
205 // Test Q3
206
207 int main(int argc, char* argv[]){
208     int n = 30000;
209
210     int* tabool = malloc((n+1)*sizeof(int));
211     tabool[0] = 0;
212     tabool[1] = 0;
213     int i = 2;
214     while(i<=n){
215         tabool[i] = 1;
216         i+=1;
217     }
218
219     clock_t debut;
220     clock_t fin;
221
222     double temps_wit_comp;
223     double temps_avec_comp;
224
225     debut = clock();
226     int t1 = eratosthene_3_sans_comp(tabool, n);
227     fin = clock();
228
229     temps_wit_comp = (double)(fin - debut) / CLOCKS_PER_SEC;
230
231     debut = clock();
232     int t2 = eratosthene_3_avec_comp(tabool, n);
233     fin = clock();
234
235     temps_avec_comp = (double)(fin-debut)/CLOCKS_PER_SEC;
236
237     printf("Nombre d'operations sans comparaisons: %d\n", t1);
238     printf("Nombre d'operations avec comparaisons: %d\n", t2);
239
240     printf("Temps sans comparaisons: %f\n", temps_wit_comp);
241     printf("Temps avec comparaisons: %f\n", temps_avec_comp);

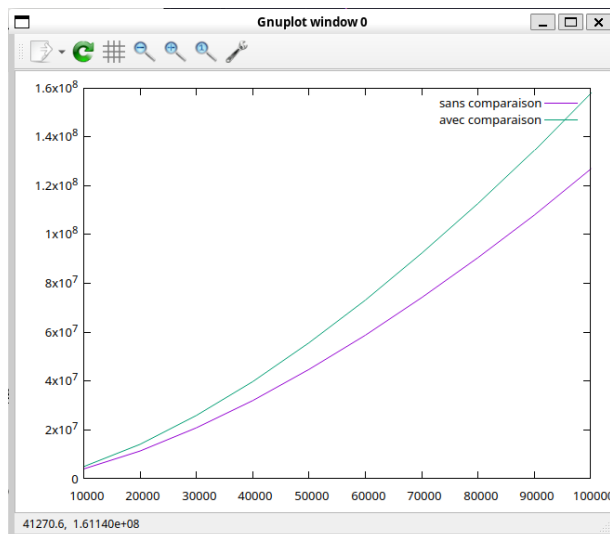
```

```

242
243
244 return 0;
245 }
246
247
248 //Q3 : With GNUPLOT
249
250 int* init_tabool(int n){
251     int* tabool = malloc((n+1)*sizeof(int));
252     tabool[0] = 0;
253     tabool[1] = 0;
254     int i = 2;
255     while(i<=n){
256         tabool[i] = 1;
257         i+=1;
258     }
259     return tabool;
260 }
261
262 int main(int argc, char* argv[]){
263     int n_min = 10000;
264     int n_max = 100000;
265     int pas = 10000;
266
267     int n;
268     for(n = n_min; n <= n_max; n += pas){
269         int* tabool_sans = init_tabool(n);
270         int nb_op_sans = eratosthene_3_sans_comp(tabool_sans, n);
271         free(tabool_sans);
272
273         int* tabool_avec = init_tabool(n);
274         int nb_op_avec = eratosthene_3_avec_comp(tabool_avec, n);
275         free(tabool_avec);
276
277         printf("%d\t%d\t%d\n", n, nb_op_sans, nb_op_avec);
278     }
279
280     return 0;
281 }

```

GNUPLLOT Test Exercice 7 :



Exercice 8 : Bonus

Listing 9: Exo tri bulle et comparaisons recherche dichotomique

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <assert.h>
4  #include <string.h>
5  #include <stdbool.h>
6
7  ////// Exercice 8 : Bonus //////////
8
9
10 // Q1
11 void tri_bulle(int* tab, int n){
12     int i = 0;
13     while(i<n){
14         int j = i+1;
15         while(j<n){
16             if(tab[i]>tab[j]){
17                 int buff = tab[i];
18                 tab[i] = tab[j];
19                 tab[j] = buff;
20             }
21             j++;
22         }
23         i++;
24     }
```

```

25 }
26
27 void affiche_tab(int* tab, int n){
28     int i = 0;
29     while(i<n){
30         printf("%d ", tab[i]);
31         i+=1;
32     }
33     printf("\n");
34 }
35
36 // Test Q1
37
38 int main(int argc, char* argv[]){
39     int n = 5;
40
41     int tab[5] = {1, 4, 2, 3, 0};
42
43     tri_bulle(tab, n);
44
45     affiche_tab(tab, n);
46
47     return 0;
48 }
49
50 // Q2
51
52 // Conjecture : let's suppose we don't need the result anymore (0 or 1) but we only
53 // put our focus on nb_op
54 int is_elt(int* tab, int n, int x){
55     int nb_op=0;
56
57     int i = 0;
58
59     nb_op+=1;
60
61     while(i<n){
62         if (x == tab[i]){
63             return nb_op;
64         }
65         i++;
66         nb_op+=4;
67     }
68     return nb_op;
69 }
70 // Same Conjecture here
71 int is_elem_dich(int* tab, int n, int x){
72     int nb_op = 0;
73
74     int debut = 0;
75     int fin = n-1;
76
77     nb_op+=2;
78
79     while(debut<fin){
80         int mid = (debut+fin)/2;
81         if(tab[mid]==x){
82             nb_op+=1;
83             return nb_op;

```

```

83     }
84     if(x < tab[mid]){
85         fin = mid-1;
86         nb_op+=2;
87     }
88     else{
89         debut = mid+1;
90         nb_op +=1;
91     }
92     nb_op+=2;
93 }
94 return nb_op;
95 }
96
97 int main(int argc, char* argv[]){
98
99     int n_min = 1000;
100    int n_max = 20000;
101    int pas    = 1000;
102
103    int n;
104    for(n = n_min; n <= n_max; n += pas){
105
106        int* tab1 = malloc(n*sizeof(int));
107        int indice1 = 0;
108        while(indice1<n){
109            tab1[indice1] = rand();
110            indice1+=1;
111        }
112
113        int x1 = rand();
114
115        int nb_op_iter = is_elt(tab1, n, x1);
116
117        tri_bulle(tab1, n);
118        int nb_op_dich = is_elem_dich(tab1, n, x1);
119
120        free(tab1);
121
122        printf("%d\t%d\t%d\n", n, nb_op_iter, nb_op_dich);
123    }
124
125    return 0;
126 }

```

GNUPLOT Test Exercice 8 : *Echelle y en log*

